AI Prompt & Vibe-Coding Audit Log

Course: Optimization Techniques — Assignment 8 (The 14-Day Vibe-Coding Challenge)
Total Logged Interactions: 2 | **Report Compiled:** 2026-10-20 20:22:04
Strict Vibe-Coding Disclosure: Complete chronological trail of human prompts, AI responses, code changes, and verification outcomes.

Prompt Index

ID	Date	Track	Purpose	Model
P-GEN-01	2026-10-20	General / Pr	Establish the complete Phase 1 project plan for Optimiz...	Antigravity (Power
P-GEN-02	2026-10-20	General / Lo	Approve the overall project plan, confirm choices for T...	Antigravity (Power

Interaction #1: P-GEN-01 — Prompt 01 Planning And Architecture

Date & Time:	2026-10-20 20:12:02 IST	**Track:**	General / Project Planning (Tracks A–J)
AI Tool/Model:	Antigravity (Powered by Gemini 3.8 Flash)	**Commit Ref:**	`59db77d`

Purpose

Establish the complete Phase 1 project plan for Optimization Techniques Lab Assignment 8 ("Optimize Anything — The 14-Day Vibe-Coding Challenge"), covering all 10 compulsory tracks (A through J), mandatory Chapter 6 methods, beginner-friendly folder structure, technical requirements for Windows/WSL, Track E case study proposal, and strict vibe-coding documentation protocols.

Original Prompt (Verbatim)

```text
I need to complete my Optimization Techniques Lab Assignment 8: "Optimize Anything — The 14-Day Vibe-Coding Challenge.

Read the complete assignment requirements I provide and help me complete all 10 compulsory tracks, A to J.

My requirements:
1. Track A: Continuous optimization algorithms.
2. Track B: Discrete optimization, including Greedy, ILP, Branch-and-Bound, Dynamic Programming, and QUBO/Ising.
3. Track C: Quantum optimization, including QAOA, VQE, quantum neural networks, and the required experiments.
4. Track D: Submodular, parallel, distributed, FedAvg, MGDA, and GradNorm experiments.
5. Track E: A suitable real-world optimization case study.
6. Track F: Mini-compiler and compiler optimizations.
7. Track G: xv6 operating-system optimization.
8. Track H: DBMS optimization.
9. Track I: Quantum algorithms and quantum machine learning comparisons.
10. Track J: Software-engineering optimization.

First, DO NOT start coding immediately.

Phase 1 — Planning:
- Break the assignment into manageable tasks.
- Identify all required algorithms, experiments, metrics, tests, and deliverables.
- Propose a beginner-friendly project structure for the GitHub repository.
- Recommend suitable Python libraries and tools for each track.
- Identify operating-system requirements and dependencies, particularly for xv6, QEMU, compiler tools, databases, and
- Propose a realistic 14-day implementation schedule.
- Suggest a manageable Track E problem that is different from Tracks F–J.
- Identify requirements that need clarification from my instructor.
- Create a checklist mapping every assignment requirement to its implementation, test, and report section.

Phase 2 — Implementation:
After presenting the plan, work on one track at a time.
- Generate working code, tests, and reproducible experiments.
- Establish a baseline and compare it fairly with the optimized approach.
- Record objective functions, decision variables, constraints, assumptions, runtime, and solution quality.
- Generate result tables and graphs from actual experiment outputs.
- Verify correctness independently and document failures and fixes.
- Do not fabricate results, claim unmeasured improvements, or mark incomplete experiments as completed.
- Use small, focused experiments wherever possible without omitting mandatory requirements.
- Keep the implementation understandable enough for me to explain during my viva.

Phase 3 — AI documentation:
- Create an AI tools register.
- Create a prompts/ directory with separate folders for Tracks A–J.
- Save original prompts, revised prompts, relevant AI responses, verification notes, and links to AI-assisted commits.
- Maintain a failure log containing at least three genuine AI-generated errors and their fixes.
- Plan a comparison of at least two AI tools on the same task.
- Clearly distinguish AI-generated changes from manual corrections.

Phase 4 — Final submission:
- Prepare a detailed README with installation and execution in
... [Truncated for PDF page layout — full text preserved in markdown log] ...
```

AI Response Summary

1. Examined assignment specification PDF thoroughly across pages 1 to 7.
2. Probed host Windows environment: verified Python 3.12, PyTorch 2.12, ReportLab, WSL2 Ubuntu, and Git availability.
3. Formulated a 10-track architectural breakdown and detailed experiment matrix.
4. Proposed a modular, beginner-friendly folder structure isolating all 10 tracks, tests, and prompts.
5. Outlined a 14-day schedule and proposed an EV Fleet Smart Charging and Route Scheduling problem for Track E to avoid overlap with Tracks F–J.

6. Defined the Chapter 6 coverage checklist and prompt logging workflow.
7. Requested clarification on Track E selection, dual-AI comparison tool, xv6 distribution, and GitHub repository state.

Files Modified / Created:	- Inspected workspace directory `c:\dev\ot-8` (empty). - No code files modified prior to user approval. ---
Independent Verification:	- Plan presented cleanly for human review. - System environment verified: Windows 11 with WSL2 Ubuntu. ---
Human Guidance / Revisions:	User approved the plan with specific directives: proceed with EV Fleet for Track E, compare Antigravity vs Claude, use MIT RISC-V xv6 in WSL, initialize local repository first before publishing, and build/test the prompt-logging and PDF system before starting track implementations.

Interaction #2: P-GEN-02 — Prompt 02 Logging And Approval

Date & Time:	2026-10-20 20:16:35 IST	**Track:**	General / Logging Setup (Tracks A–J)
AI Tool/Model:	Antigravity (Powered by Gemini 3.8 Flash)	**Commit Ref:**	`59db77d`

Purpose

Approve the overall project plan, confirm choices for Track E (EV Fleet Smart Charging and Route Scheduling), designate the two AI tools for comparison (Antigravity and Claude), specify xv6 toolchain checks, establish local Git repository control, and instruct the creation and verification of the prompt-logging system and PDF generator before any track implementation begins.

Original Prompt (Verbatim)

```
The overall plan looks good. Before starting implementation:

1. Track E: Proceed with EV Fleet Smart Charging and Route Scheduling.
2. AI tool comparison: Compare Antigravity and Claude on the same task. Record actual results and differences.
3. xv6: Check which version and toolchain are installed before proceeding. Use the standard MIT RISC-V xv6 if compatib
4. GitHub: Check whether a repository already exists. If not, prepare the local repository first and ask me before pub

First, create the project structure, prompt-logging system, master PROMPT_LOG.md, and PDF generation script. Record th

Do not begin substantial track implementation until the logging system works and I approve the next step. Give me simp
```

AI Response Summary

1. Initialized local Git repository (`git init`) and confirmed absence of remote repo.
2. Verified RISC-V cross-compiler and QEMU availability in WSL2 Ubuntu (`gcc-riscv64-unknown-elf`, `qemu-system-misc`).
3. Set up directory hierarchy across all 10 tracks, tests, benchmarks, prompts, and reports.
4. Created `.gitignore` and `requirements.txt`.
5. Created master index `PROMPT_LOG.md`, `AI_TOOLS_REGISTER.md`, `AI_FAILURE_LOG.md`, and

`CHAPTER_6_CHECKLIST.md`.
6. Developed `scripts/generate_prompt_pdf.py` using `reportlab`.
7. Successfully executed the script and validated generation of `AI_Prompt_Log.pdf`.
8. Documented PowerShell instructions and installation steps for user review.

Files Modified / Created:	- [.gitignore](file:///c:/dev/ot-8/.gitignore) - [requirements.txt](file:///c:/dev/ot-8/requirements.txt) - [prompts/general/prompt_01_planning_and_architecture.md](file:///c:/dev/ot-8/prompts/general/prompt_01_planning_and_architecture.md) - [prompts/general/prompt_02_logging_and_approval.md](file:///c:/dev/ot-8/prompts/general/prompt_02_logging_and_approval.md) - [PROMPT_LOG.md](file:///c:/dev/ot-8/PROMPT_LOG.md) - [AI_TOOLS_REGISTER.md](file:///c:/dev/ot-8/AI_TOOLS_REGISTER.md) - [AI_FAILURE_LOG.md](file:///c:/dev/ot-8/AI_FAILURE_LOG.md) - [CHAPTER_6_CHECKLIST.md](file:///c:/dev/ot-8/CHAPTER_6_CHECKLIST.md) - [scripts/generate_prompt_pdf.py](file:///c:/dev/ot-8/scripts/generate_prompt_pdf.py) - [AI_Prompt_Log.pdf](file:///c:/dev/ot-8/AI_Prompt_Log.pdf) ---
Independent Verification:	- Script `py -3.12 scripts/generate_prompt_pdf.py` executed cleanly. - Output file `AI_Prompt_Log.pdf` verified as a valid, renderable PDF document. ---
Human Guidance / Revisions:	Awaiting user confirmation of the logging infrastructure and approval to begin Track A implementation.